

Tick Notes — architecture, 5-day build plan, and database choice

Operational notes (quick recap)

Closed testing + new app bundle. Uploading new builds with new features to the same closed-testing track does not reset or break your testing phase. The "12 testers for 14 consecutive days" clock tracks *tester opt-in continuity*, not whether your app's code stays frozen. As long as you keep the same `(applicationId)`/package name, keep using the same testing track, and increase `(versionCode)` on every upload, you can ship as many builds as you want during the 14-day window — this is normal and expected. What *would* disrupt it: removing all testers and restarting the track, changing the package name, or getting the release rejected/suspended. So: build the new features, upload a new AAB to the existing closed-testing track whenever ready, and the existing testing progress keeps counting.

Renaming the GitHub repo. Repo → Settings → General → "Repository name" field at the top → type `(tick_notes)` → Rename. GitHub keeps a redirect from the old URL for a while, but update your local clone anyway:

```
git remote set-url origin https://github.com/Maaz-saqib/tick_notes.git
git remote -v
```

This is purely cosmetic and unrelated to your Play Store listing. Two things it does *not* touch: the Flutter project's internal package name (`(pubspec.yaml)` → `(name:)`, plus Android `(applicationId)` / iOS bundle id) is a separate, much riskier rename — don't touch that mid-closed-testing, since the `applicationId` is permanent once an app is published. You can freely rename the *display name* shown to users on the Play Store listing at any time without consequence.

Architecture: MVVM + repository layer

Two real options here: full Clean Architecture (presentation → domain with use-case classes → data) or MVVM with a repository layer. For a solo developer shipping three feature areas in days rather than months, **MVVM with a repository layer** is the better fit, and it also happens to be what Flutter's own team currently recommends in their app-architecture guide (View → ViewModel → Repository → Service, organized feature-first rather than type-first).

- **View** — your widgets. Dumb. Only reads state from a ViewModel and calls its methods.

- **ViewModel** — a Riverpod `Notifier`/`AsyncNotifier` per feature (`NotesViewModel`, `TodoViewModel`, `PomodoroViewModel`, `AnalyticsViewModel`). Holds UI state, talks to repositories, contains feature logic.
- **Repository** — one per feature, the single source of truth. Wraps the local database (and, later, an optional remote source) behind a clean interface so the ViewModel never touches SQL or Firebase directly.
- **Service** — small shared helpers used by more than one feature: notification scheduling, sharing, theme persistence.

Full Clean Architecture's extra `domain` layer (a use-case class for every single action — `AddNoteUseCase`, `DeleteNoteUseCase`, etc.) buys you more testability and team-scalability, but for one developer building three CRUD-ish features in five days it's mostly ceremony. Keep the door open: if `tick_notes` grows, gets contributors, or you start reusing logic across features in nontrivial ways, lifting shared logic into a thin `domain/` folder later is a small, incremental refactor — not a rewrite.

Suggested folder structure (feature-first):

```
lib/  
  core/  
    theme/          # ThemeData, ThemeNotifier, color seeds  
    database/       # Drift database + tables (notes, todos,  
pomodoro_sessions, settings)  
    notifications/  # flutter_local_notifications wrapper service  
  common/  
    widgets/        # shared buttons, empty states, color picker, etc.  
  features/  
    notes/  
      notes_repository.dart  
      notes_view_model.dart  
      notes_list_screen.dart  
      note_editor_screen.dart  
    todo/  
      todo_repository.dart  
      todo_view_model.dart  
      todo_list_screen.dart  
      todo_editor_screen.dart  
    pomodoro/  
      pomodoro_repository.dart  
      pomodoro_view_model.dart  
      pomodoro_screen.dart  
    analytics/  
      analytics_view_model.dart  
      analytics_screen.dart  
  main.dart
```

A minimal skeleton of the pattern, so the shape is concrete:

```
dart
```

```

// core/database/app_database.dart
@DriftDatabase(tables: [Notes, Todos, PomodoroSessions, Settings])
class AppDatabase extends _$AppDatabase {
  AppDatabase() : super(_openConnection());
}

class Notes extends Table {
  IntColumn get id => integer().autoIncrement(());
  TextColumn get title => text();
  TextColumn get body => text();
  IntColumn get colorTag => integer().withDefault(const Constant(0))();
  DateTimeColumn get updatedAt => dateTime();
}

// features/notes/notes_repository.dart
class NotesRepository {
  NotesRepository(this._db);
  final AppDatabase _db;

  Stream<List<Note>> watchAll() => _db.select(_db.notes).watch();
  Future<int> add(NotesCompanion note) => _db.into(_db.notes).insert(note);
  Future<void> delete(int id) =>
    (_db.delete(_db.notes)..where((t) => t.id.equals(id))).go();
}

// features/notes/notes_view_model.dart
@riverpod
class NotesViewModel extends _$NotesViewModel {
  @override
  Stream<List<Note>> build() => ref.watch(notesRepositoryProvider).watchAll();

  Future<void> addNote(String title, String body, int colorTag) {
    return ref.read(notesRepositoryProvider).add(
      NotesCompanion.insert(title: title, body: body, colorTag: Value(colorTag))
    );
  }
}

```

Todo and Pomodoro follow the exact same shape — that repetition is the point; once you've built Notes on day 2, Todo on day 3 is mostly copy-adapt.

The 5-day plan

Each day is roughly: 1-2 hours reading/watching the listed resource, then implement. Order matters — day 1 is the foundation everything else sits on.

Day 1 — Foundation: architecture, local database, theming

Learn:

- Flutter's official app-architecture guide: docs.flutter.dev/app-architecture (the View/ViewModel/Repository/Service breakdown above is taken directly from here)
- Riverpod basics, specifically `Notifier`/`AsyncNotifier` and code generation: riverpod.dev → "Concepts" section
- Drift "Getting started" + table/DAO definitions: drift.simonbinder.dev

Build:

- Add dependencies: `riverpod`, `riverpod_generator`, `drift`, `drift_dev`, `sqlite3_flutter_libs`, `path_provider`, `share_plus`, `flutter_local_notifications`, `timezone`, `fl_chart`
- Set up the feature-first folder structure above
- Define the Drift schema: `notes`, `todos`, `pomodoro_sessions`, `settings` tables
- Migrate your existing notes feature's data layer onto Drift + the repository pattern (this is the one piece of "refactor" work — everything after this is net-new)
- Build the app-wide theme system: a `ThemeNotifier` (Riverpod) backed by Material 3 `ColorScheme.fromSeed`, with the selected seed color and light/dark mode persisted in the `settings` table

Day 2 — Notes feature, complete

Learn:

- `share_plus` docs on pub.dev (text + file sharing)
- Material 3 component guidance for cards, FAB, bottom sheets: m3.material.io

Build:

- Notes list (grid view), search/filter
- Create/edit screen: title, body, per-note color tag picker
- Delete via swipe-to-dismiss with an undo snackbar
- Share a note as plain text via `share_plus`
- Empty-state illustration when there are zero notes

- Wire `NotesViewModel` → `NotesRepository` → Drift, following Day 1's skeleton

Because everything routes through Drift (a local SQLite file), this feature is offline by construction — no network call exists in the path.

Day 3 — Todo feature

Learn:

- `flutter_local_notifications` + `timezone` setup for exact-time scheduled notifications (pub.dev README covers Android 13+ permission requests and iOS authorization)

Build:

- Todo list grouped into Today / Upcoming / Done
- Add/edit screen: title, due date+time, reminder toggle
- On save, schedule a local notification via `flutter_local_notifications.zonedSchedule` at the due time
- Mark-complete animation, swipe-delete with undo, per-item color tag, share a single todo or the whole list as text
- Same ViewModel → Repository → Drift wiring as notes

Day 4 — Pomodoro feature, two modes

Learn:

- The "store a timestamp, compute elapsed on rebuild" pattern for timers that survive backgrounding (a plain `Timer.periodic` pauses when the app is backgrounded on some platforms — instead persist `sessionStart` + `targetDuration`, and recompute remaining time whenever the widget rebuilds or the app resumes)
- `flutter_local_notifications.zonedSchedule` again, this time to fire the "session complete" alert at the exact end time regardless of whether the app is foregrounded

Build:

- Mode 1: Pomodoro — 25 min focus / 5 min break, auto-cycling, both durations configurable in settings
- Mode 2: Open — counts up from zero indefinitely
- Start / pause / stop controls, usable at any point in either mode
- On stop or natural completion, persist the session (start time, end time, duration, mode) into the `pomodoro_sessions` table — this is the data Day 5's chart reads
- Circular countdown/count-up UI

Day 5 — Analytics, polish, and release

Learn:

- `fl_chart`'s `BarChart` — pub.dev README and example app
- Drift's `groupBy` / aggregate query support for "sum focused minutes per day, last 30 days"

Build:

- Analytics screen: a 30-day bar chart of focused minutes, querying `pomodoro_sessions` grouped by day (fill any day with zero sessions as a zero-height bar so the 30-day axis stays continuous)
- Tap a bar to see that day's individual sessions
- Final visual pass: consistent spacing/typography across all three features, Material 3 `ColorScheme.fromSeed` applied everywhere
- Offline pass: toggle airplane mode and verify notes, todos, the timer, and the analytics chart all keep working — they should, since nothing in the path above touches the network
- Bump `versionCode` / `versionName`, `flutter build appbundle`, upload to the existing closed-testing track, push to the renamed `tick_notes` repo

Database: local store + optional cloud sync

	Free option	Paid / scaling option
Local (offline) store	Drift (SQLite wrapper) — actively maintained, mature, free and open-source. Good fit here because the 30-day analytics chart needs <code>GROUP BY</code> -style aggregate queries, which Drift's SQL foundation handles naturally.	— (Drift itself has no paid tier; the "paid" axis only shows up if you add cloud sync, below)
Local store, alternative	Isar (NoSQL) — fast and pleasant API, but flagged: the original maintainer stepped away and it's now community-forked, which is a real risk for a project you intend to maintain long-term. Hive has the same maintenance problem. For a new project in 2026, Drift is the safer default.	—

	Free option	Paid / scaling option
Cloud backup / cross-device sync (optional, for later)	Firebase Firestore Spark plan (free quota) — you already have <code>firebase.json</code> in the repo, so this is the path of least resistance if you want notes/todos to sync across a user's devices.	Firebase Blaze (pay-as-you-go once you exceed the free quota), or Supabase Pro (~\$25/mo, Postgres-based, open-source alternative to Firebase) if you'd rather not be on Firebase, or ObjectBox Sync / PowerSync if you want purpose-built offline-first sync layered on top of a local SQLite store.

Recommendation for this project: ship with **Drift only**. It's local, free, requires no internet connection at all, and is enough for a single-device personal-productivity app. Keep Firebase in the repo as an optional add-on for a future "sync across my devices" feature — don't build that now, since it adds real complexity (conflict resolution, auth) for a feature you haven't asked for yet.